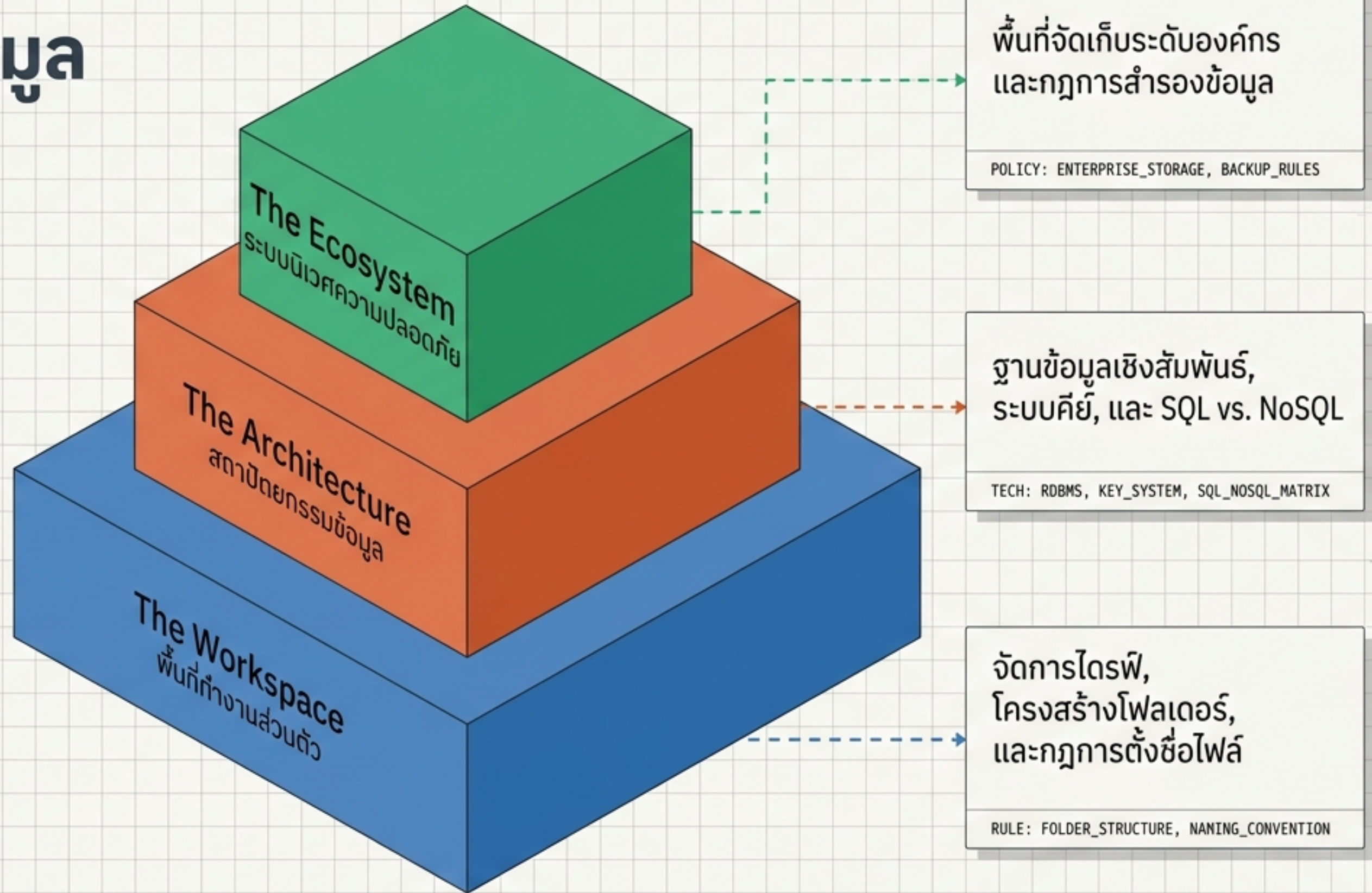


ปรมาจารย์ด้าน การจัดการข้อมูล

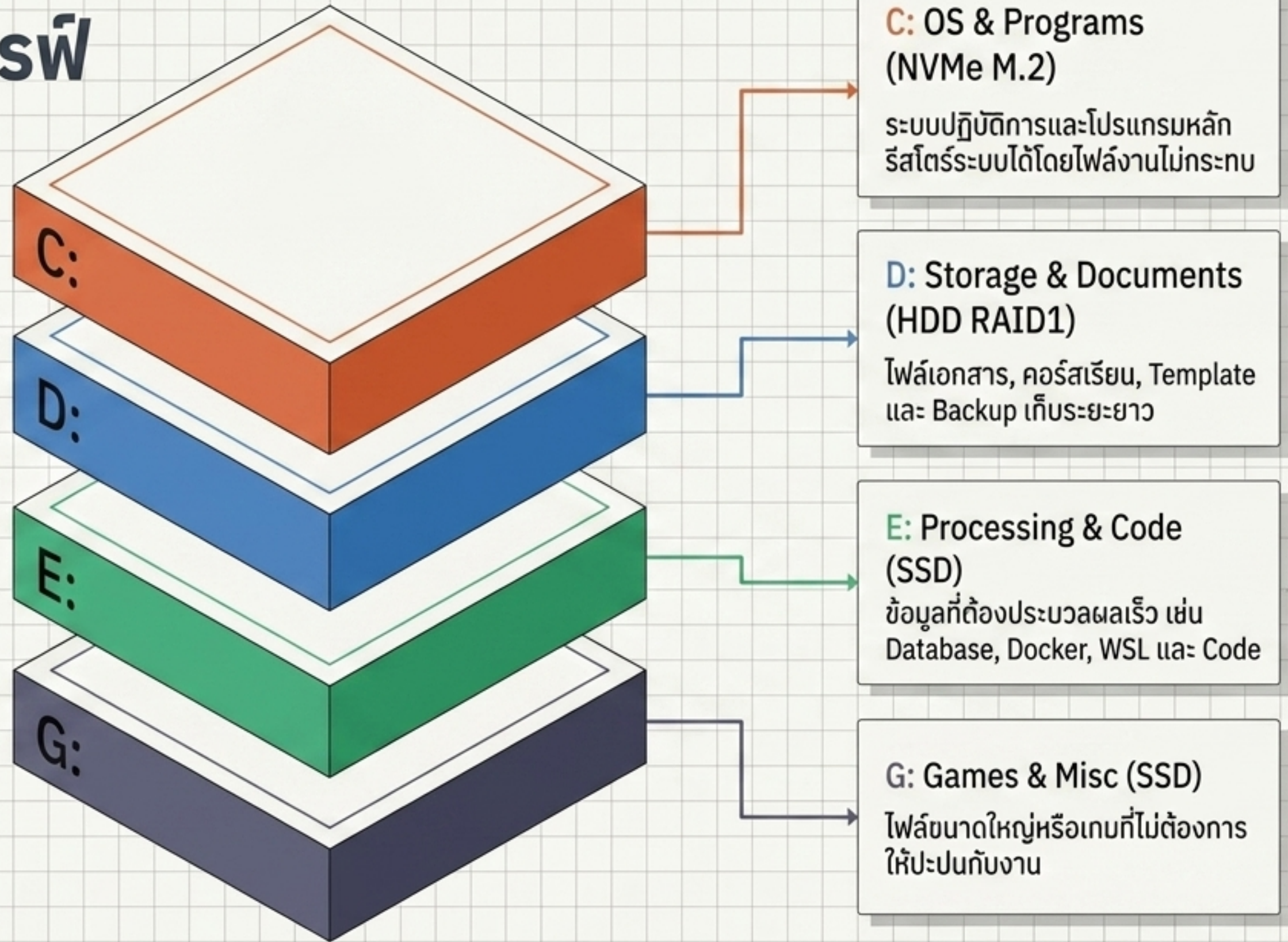
// จากระบบไฟล์ส่วนตัว สู่สถาปัตยกรรมระดับองค์กร

คู่มือฉบับสมบูรณ์สำหรับนักพัฒนาและผู้เชี่ยวชาญไอที

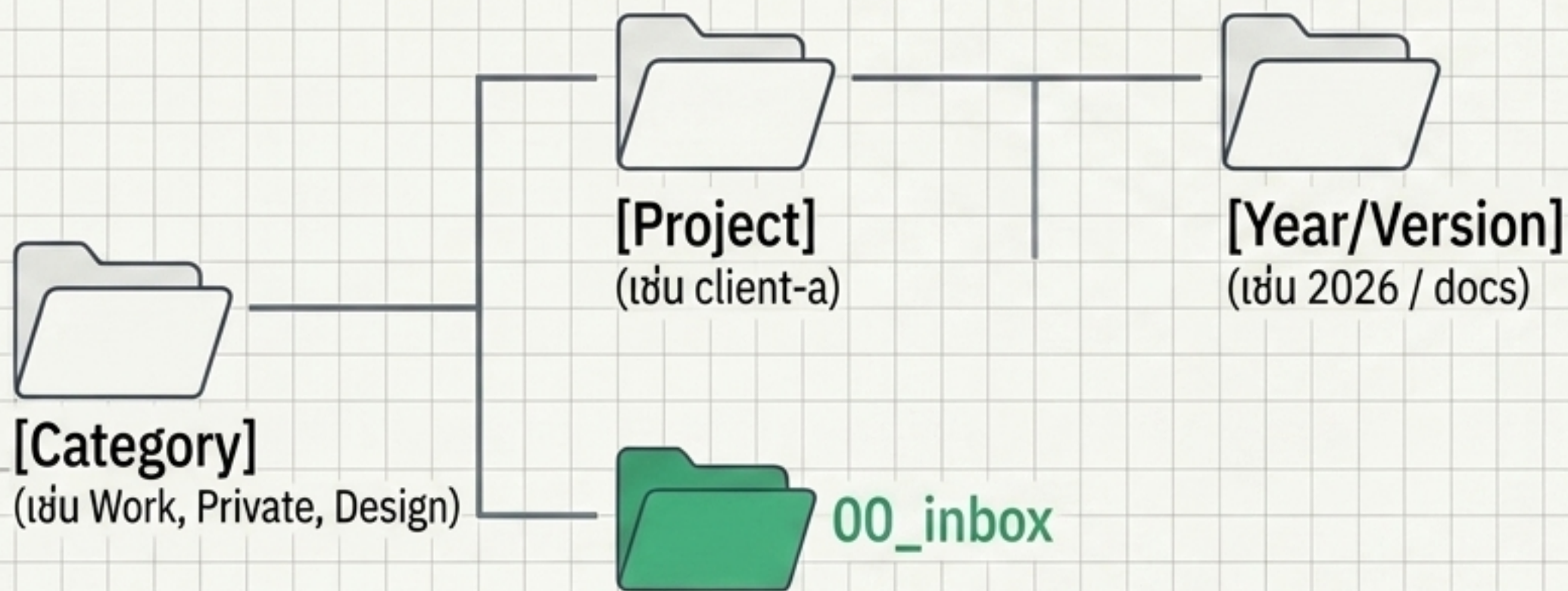
3 เสาหลักสู่ความ เป็นเลิศด้านข้อมูล



จัดการฮาร์ดแวร์ ด้วยยุทธศาสตร์ 4 ไดรฟ์



โครงสร้างโฟลเดอร์ ที่ค้นหาได้ในพริบตา



Rule 1: Lowercase & Hyphens

ใช้ตัวพิมพ์เล็กและขีดกลางเสมอ
เช่น project-x ไม่เว้นวรรค

Rule 2: The _inbox Rule

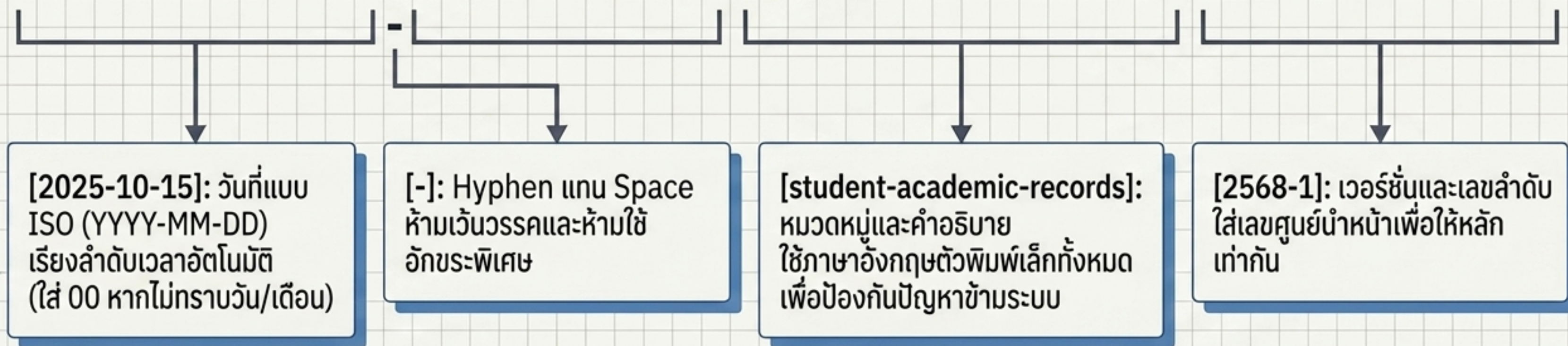
สร้างโฟลเดอร์ **00_inbox** ไว้รับไฟล์ชั่วคราว
แล้วเคลียร์เข้าหมวดหมู่ทุกสัปดาห์

Rule 3: Reusable Folders

สร้างโฟลเดอร์มาตรฐาน (Template, ISO, Software) ไว้ใช้ซ้ำกับทุกโปรเจกต์

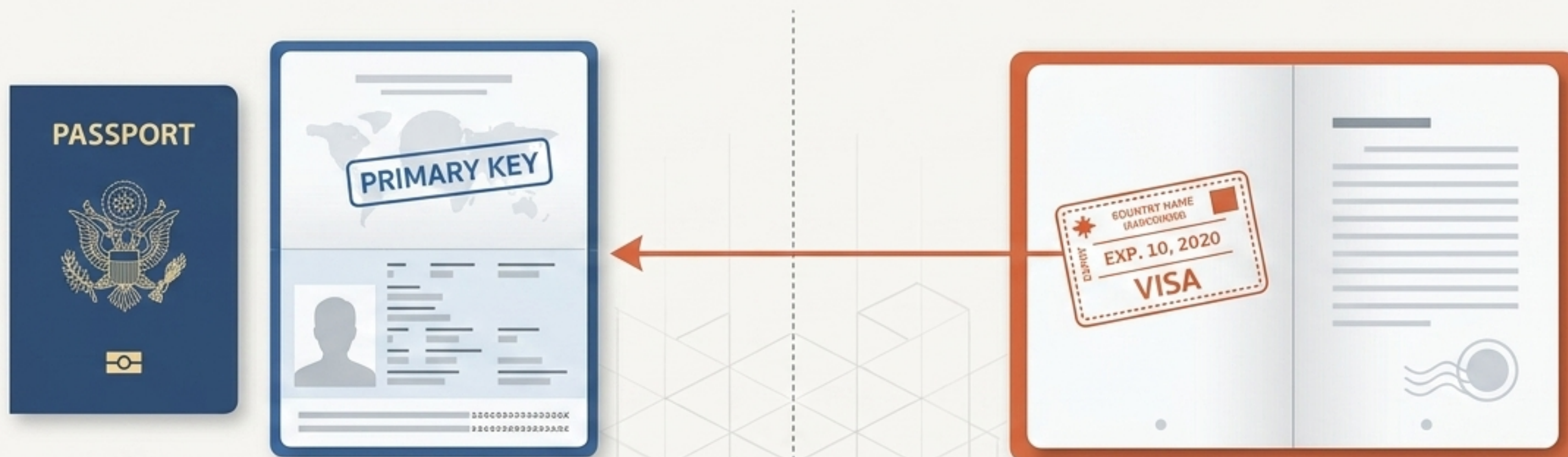
คำแนะนำโครงสร้างชื่อไฟล์ที่สมบูรณ์แบบ

2025-10-15-student-academic-records-2568-1.xlsx



! Windows จำกัด Path + ชื่อไฟล์ที่ 260 ตัวอักษร ให้ตั้งชื่อสั้นแต่ได้ใจความ (25-30 ตัวอักษร)

ทำความเข้าใจระบบคีย์ด้วยการพาสปอร์ตและวีซ่า



Primary Key (คีย์หลัก)

- **เปรียบเสมือนเลขพาสปอร์ต:** ระบุดัชนีของข้อมูลในตารางนั้นๆ ได้อย่างเฉพาะเจาะจง (Uniqueness)
- **คุณสมบัติ:** ห้ามซ้ำกันเด็ดขาด และห้ามเป็นค่าว่าง (Not Null / Entity Integrity)

Foreign Key (คีย์นอก)

- **เปรียบเสมือนวีซ่า:** เชื่อมโยงตัวตนของคุณเข้ากับระบบของประเทศอื่น
- **คุณสมบัติ:** วีซ่าจะใช้งานได้ก็ต่อเมื่อพาสปอร์ตนั้นมีอยู่จริง (Referential Integrity) สามารถมีค่าซ้ำได้

เปรียบเทียบบทบาทและการรักษาความถูกต้องของข้อมูล

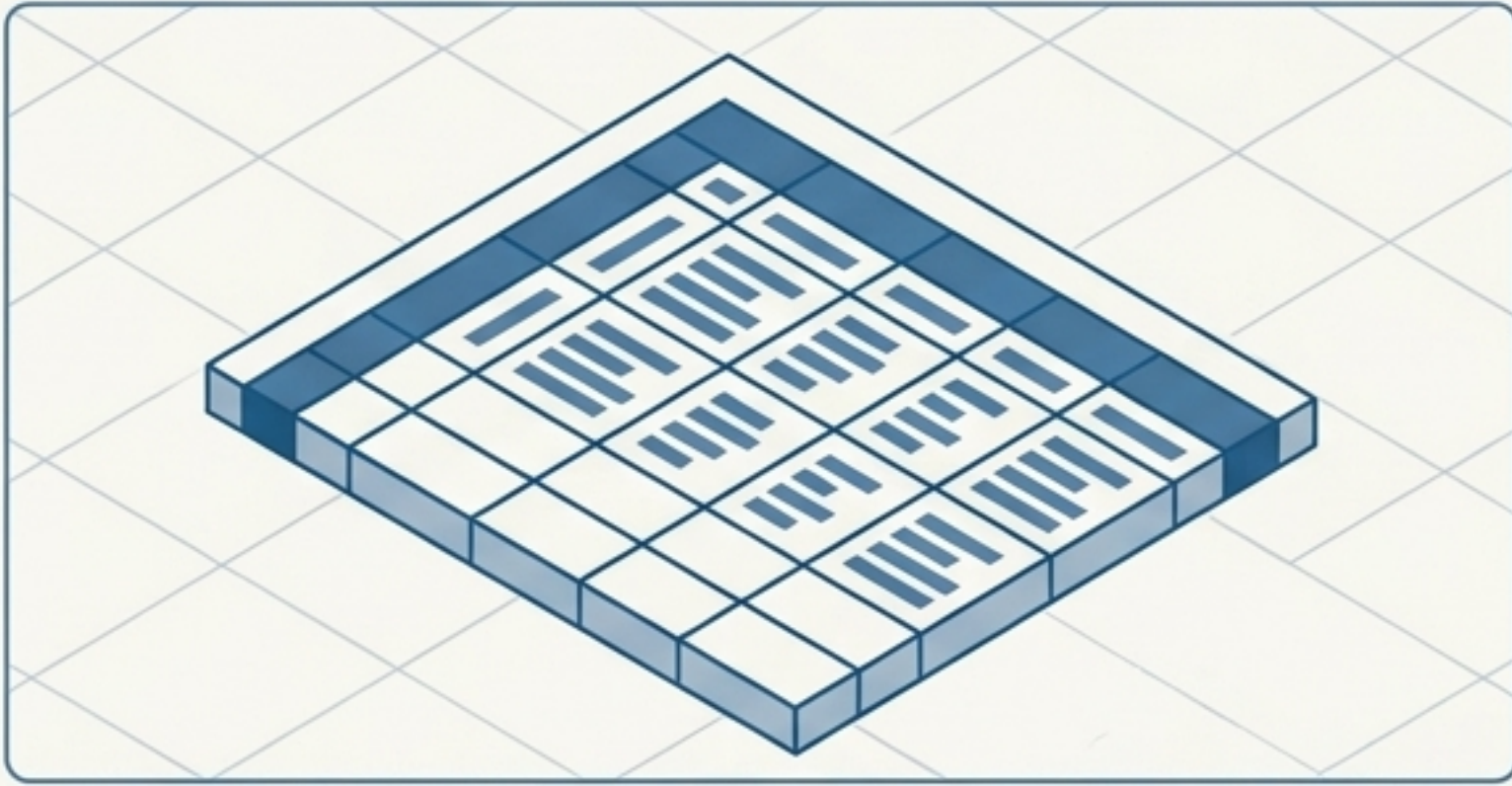
คุณสมบัติ	Primary Key	Foreign Key
หน้าที่ (Role)	ระบุข้อมูลในตารางตัวเอง	สร้างความสัมพันธ์ระหว่าง 2 ตาราง
ความซ้ำซ้อน (Uniqueness)	ซ้ำไม่ได้	ซ้ำได้ (มีได้หลายรายการ)
ค่าว่าง (Nullability)	ห้ามว่าง	ว่างได้ (หากความสัมพันธ์ไม่บังคับ)

เมื่อลบ/แก้ไข Primary Key จะเกิดอะไรขึ้นกับ Foreign Key? (Referential Actions)



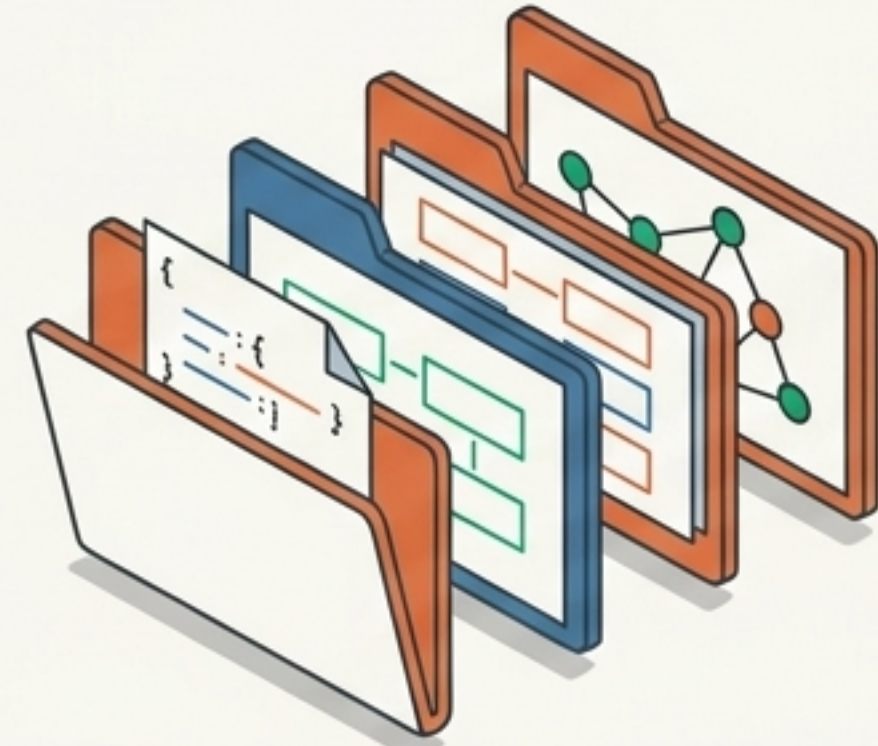
SQL vs NoSQL: สมุดบัญชีที่เส้น กับ ลื่นชักเอกสารอิสระ

SQL (Relational)



- เปรียบเสมือนสมุดบัญชีที่มีเส้นตารางชัดเจน ข้อมูลต้องอยู่ในโครงสร้าง Schema ที่กำหนด
- จุดเด่น: ACID Compliance (ปลอดภัยสูงสุด), Complex JOINS

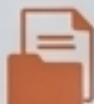
NoSQL (Not Only SQL)



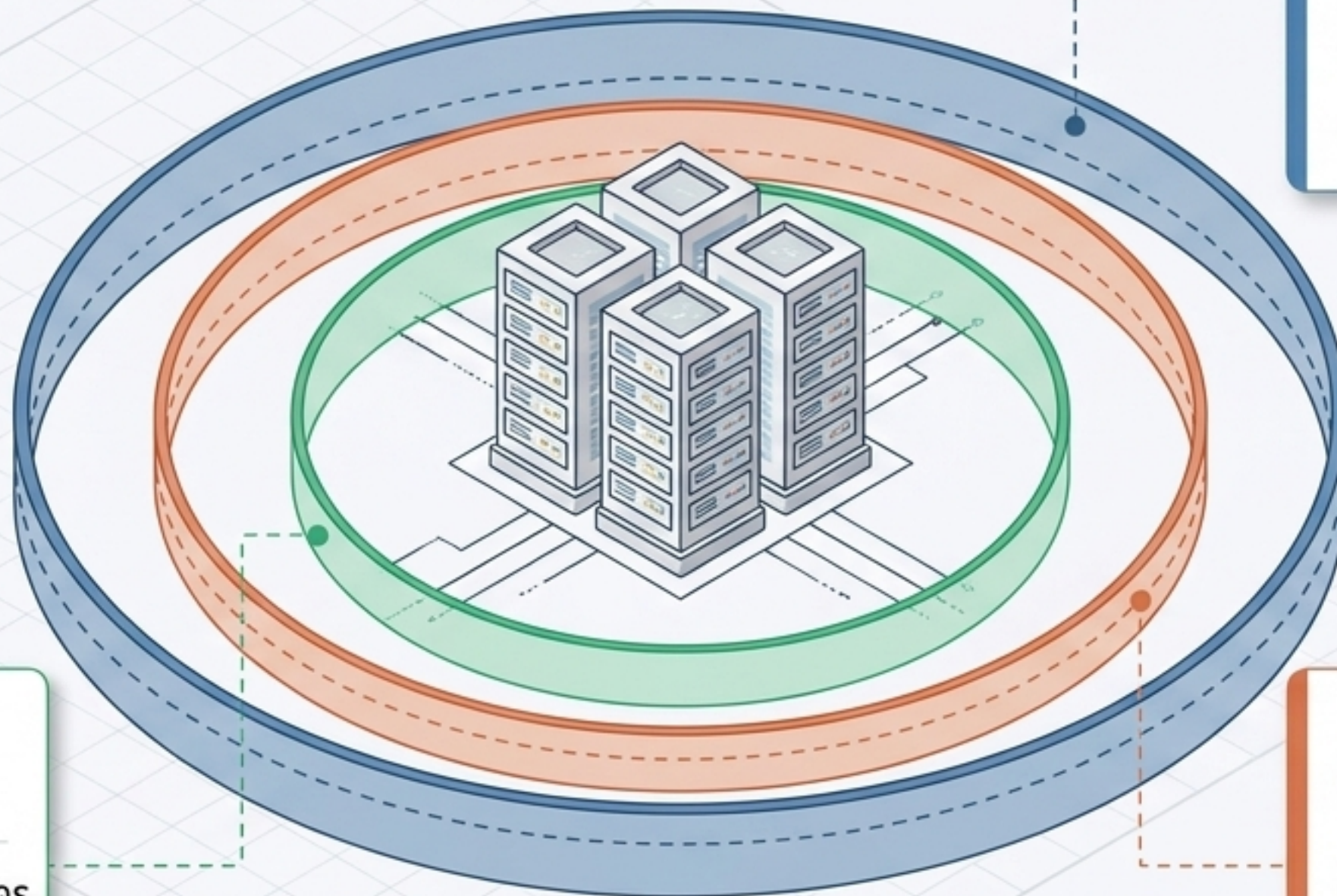
- เปรียบเสมือนลิ้นชักที่แต่ละแฟ้มมีจำนวนเอกสารไม่เท่ากัน ยืดหยุ่น ไร้ข้อจำกัด
- ประเภทหลัก: Document (MongoDB), Key-Value (Redis), Column-Family, Graph

Market Insight Callout: 🏆 PostgreSQL ครองอันดับ 1 (SO Survey 2025: 55.6%)
เป็น 'Swiss Army Knife' ที่รองรับทั้ง SQL และ JSON ใช้งานได้ครบจบในตัวเอง

เมทริกซ์ตัดสินใจ: เลือกฐานข้อมูลให้ตรงกับสถาปัตยกรรม

Use Case	Recommended DB	Reason
E-Commerce & Payment	 SQL (PostgreSQL)	ต้องใช้ ACID การันตีว่าโอนเงินแล้วข้อมูลไม่หาย
Social Media / Content	 NoSQL (MongoDB)	โครงสร้าง User-Generated Content เปลี่ยนแปลงบ่อย ต้องการความยืดหยุ่น
Chat / Messaging App	 NoSQL (Mongo + Redis)	มีข้อความมหาศาล ต้องการเขียนเร็ว และดึงข้อมูล Real-Time
IoT / Sensor Data	 NoSQL (Cassandra)	Write-Heavy ข้อมูลมหาศาล ต้อง Scale แนวนอน
Recommendation Engine	 Graph DB (Neo4j)	จัดการข้อมูลความสัมพันธ์ที่ซับซ้อนและซ้อนทับกันหลายชั้น

ความปลอดภัยและการจัดเก็บ ข้อมูลระดับองค์กร



Shield 1: Cloud Storage & DBMS

การเก็บข้อมูลไว้บนคลาวด์ เพิ่มความสามารถในการเข้าถึงข้อมูลทุกที่ทุกเวลา พร้อมขยายสเกลอัตโนมัติ

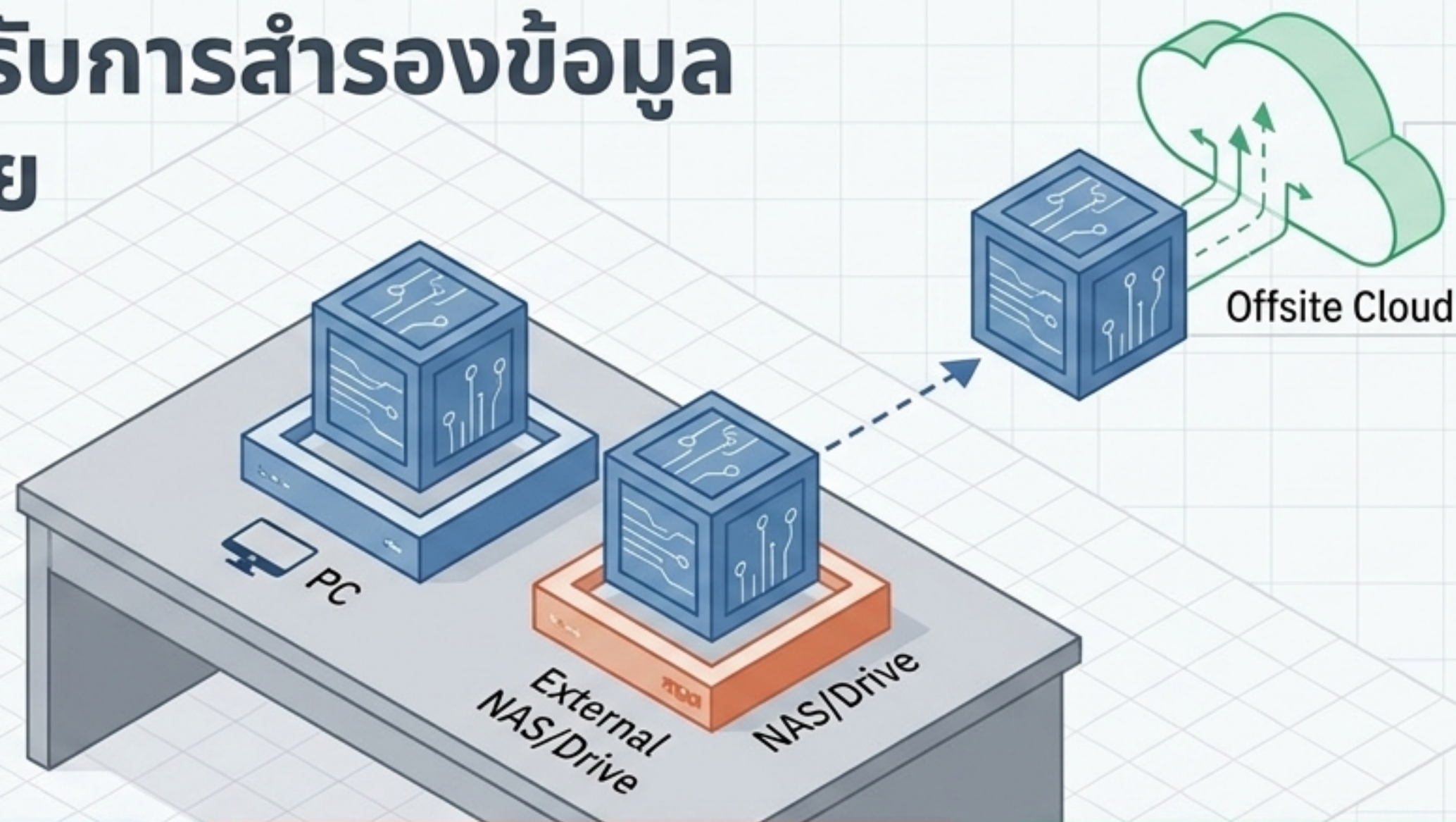
Shield 3: Compliance (PDPA)

สถาปัตยกรรมต้องรองรับกฎหมายคุ้มครองข้อมูลส่วนบุคคล การเก็บ log เข้าถึง และสิทธิการลบข้อมูล

Shield 2: Access Control & Encryption

ระบบการยืนยันตัวตน (Authentication) และการเข้ารหัสป้องกันบุคคลภายนอกเข้าถึงข้อมูลความลับ

กฎ 3-2-1 สำหรับการสำรองข้อมูล ที่ไม่มีวันสูญหาย



[3] สำเนาข้อมูล (Copies)

มีข้อมูลหลัก 1 ชุด และข้อมูลสำรองอีก 2 ชุด
(ป้องกันไฟล์เสียหาย)

[2] สื่อที่ต่างกัน (Different Media)

เก็บข้อมูลในอุปกรณ์ต่างชนิดกัน
(เช่น แฟ้มที่อป 1 เครื่อง + NAS 1 เครื่อง)

[1] นอกสถานที่ (Offsite/Cloud)

เก็บสำเนา 1 ชุดไว้บนนอกสถานที่
ป้องกันภัยพิบัติทางกายภาพ



Pro Tip: ใช้ระบบ Auto-Sync เพื่อให้ข้อมูลอัปเดตอัตโนมัติ

สถาปัตยกรรม Polyglot Persistence ในโลกจริง

75% ของ Startup ใช้ฐานข้อมูลหลาย
ประเภทผสมผสานกัน (HTAP)
เพื่อรีดประสิทธิภาพสูงสุด



PostgreSQL (SQL)

จัดการธุรกรรมการเงินและ Order
ที่ต้องการความแม่นยำสูงสุด (ACID)

MongoDB (Document)

เก็บ Activity Logs และ User Content
ที่โครงสร้างไม่ตายตัว

Redis (Key-Value)

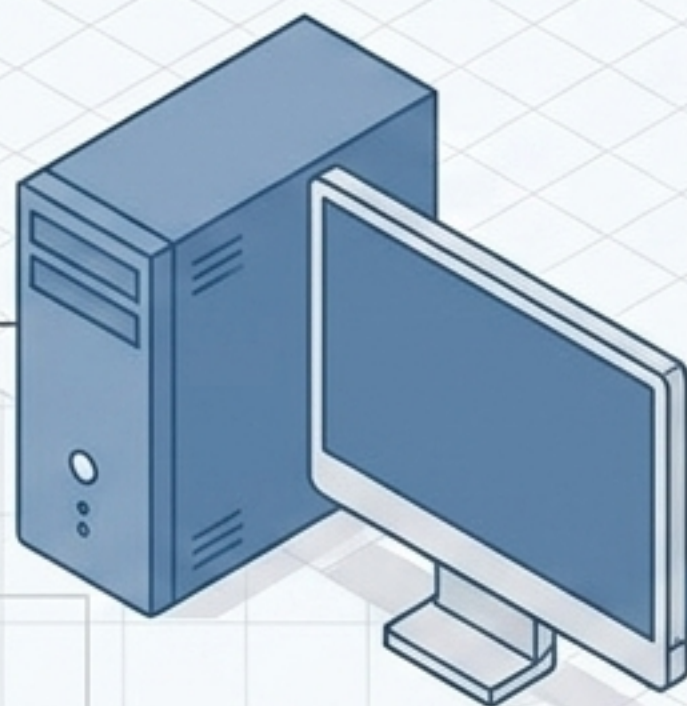
เก็บข้อมูล Session และ Cache
ที่ต้องการดึงข้อมูลระดับเสี้ยววินาที

Elasticsearch

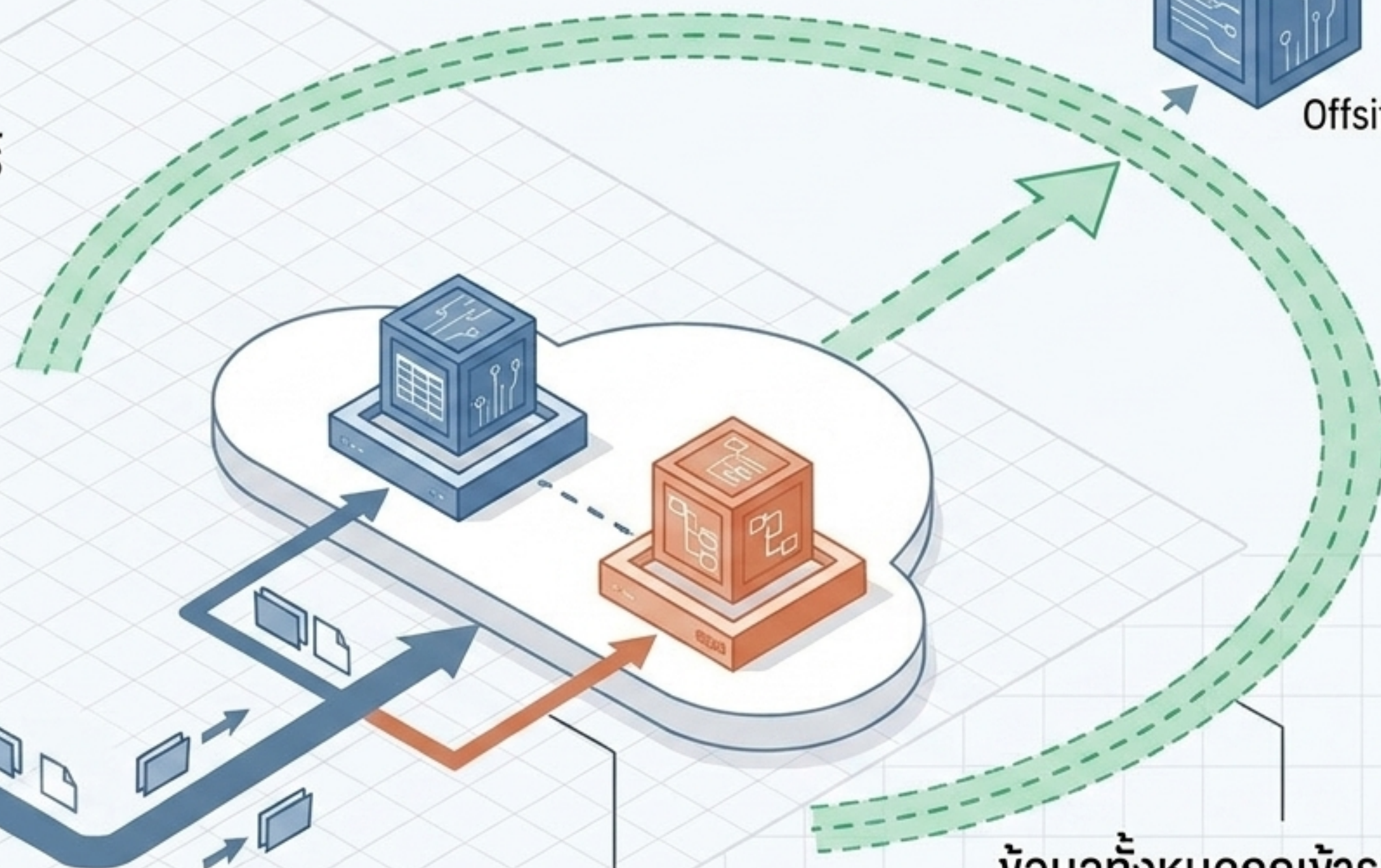
สำหรับการค้นหาแบบ Full-Text Search

พิมพ์เขียวระบบนิเวศข้อมูล ที่สมบูรณ์แบบ

ไฟล์และโค้ดถูกจัดการด้วยยุทธศาสตร์
4 ไดรฟ์ (C/D/E/G)
และโฟลเดอร์ที่เป็นระบบ



Developer's PC



Offsite Cloud

ข้อมูลถูกเขียนลงสถาปัตยกรรม
Polyglot ที่เลือก Database ตรงกับบริบท

ข้อมูลทั้งหมดถูกเข้ารหัสและ
ทำสำเนา Sync ขึ้น Cloud
ตามกฎ 3-2-1 อย่างอัตโนมัติ

เช็คลิสต์ตรวจสอบสถาปัตยกรรมข้อมูลของคุณ

- ✓ ไฟล์งานและโฟลเดอร์ถูกแยกออกจากไดรฟ์ OS (C:) อย่างเด็ดขาด
- ✓ ชื่อไฟล์ใช้ YYYY-MM-DD, ตัวพิมพ์เล็ก, และขีดกลาง (Hyphen) เสมอ
- ✓ มีโฟลเดอร์ 00_inbox สำหรับจัดการไฟล์ชั่วคราว
- ✓ เข้าใจความสัมพันธ์ของข้อมูล (Primary Key / Foreign Key) และข้อจำกัด (Integrity)
- ✓ เลือกใช้ Database ได้ถูกต้อง (SQL สำหรับธุรกรรม, NoSQL สำหรับความยืดหยุ่น)
- ✓ ระบบมี Backup อัตโนมัติตามกฎ 3-2-1 (3 สำเนา, 2 อุปกรณ์, 1 คลาวด์ออฟไซต์)

“สถาปัตยกรรมที่ดี ไม่ได้แปลว่าซับซ้อนที่สุด แต่คือระบบที่จัดการได้ง่าย คาดเดาได้ และปลอดภัยที่สุด”